

## Demonstration: Using a Transaction

In this demonstration, you will see how to use a transaction.

### Demonstration Steps

Use a Transaction

1. If you did not complete the previous demonstrations in this module, ensure that the **20767C-MIA-DC** and **20767C-MIA-SQL** virtual machines are both running, and log on to **20767C-MIA-SQL** as **ADVENTUREWORKS\Student** with the password **Pa55w.rd**.
2. In the **D:\Demofiles\Mod07** folder, run **Setup.cmd** as administrator.
3. Start **SQL Server Management Studio** and connect to the **MIA-SQL** database engine instance using Windows authentication.
4. In Object Explorer, expand **Databases**, expand **DemoDW**, and then expand **Tables**.
5. Right-click **dbo.StagingTable** and click **Select Top 1000 Rows** to verify that it contains product data.
6. Right-click **dbo.ProductionTable** and click **Select Top 1000 Rows** to verify that it is empty.
7. Start **SQL Server Data Tools** and open the **Transactions.sln** solution in the **D:\Demofiles\Mod07** folder.
8. In Solution Explorer, double-click **Move Products.dtsx**. Note that the control flow consists of a Data Flow task named **Copy Products** that moves products from a staging table to a production table, and a **SQL Command** task named **Update Prices** that sets the product price.
9. On the **Debug** menu, click **Start Debugging** to run the package and note that the **Update Prices** task fails. Then on the **Debug** menu, click **Stop Debugging**.
10. In **SQL Server Management Studio**, select the top 1,000 rows from the **dbo.ProductionTable** table, noting that it now contains product data but the prices are all set to 0.00. You want to avoid having products with invalid prices in the production table, so you need to modify the SSIS package to ensure that, when the price update task fails, the production table remains empty.
11. On the **File** menu, point to **New**, and then click **Query with Current Connection**, type the following Transact-SQL code, and then click **Execute**. This deletes all rows in the **dbo.ProductionTable** table:  

```
TRUNCATE TABLE DemoDW.dbo.ProductionTable;
```
12. In **SQL Server Data Tools**, click anywhere on the Control Flow surface and press F4.
13. In the Properties pane, set the **TransactionOption** property to **Required**.
14. Click the **Copy Products** task, and in the Properties pane, set the **FailPackageOnFailure** property to **True** and ensure the **TransactionOption** property is set to **Supported**.
15. Repeat the previous step for the **Update Prices** task.
16. Run the package and note that the **Update Prices** task fails again. Stop debugging.
17. In **SQL Server Management Studio**, select the top 1,000 rows from the **dbo.ProductionTable** table, noting that it is empty, even though the **Copy Products** task succeeded. The transaction has rolled back the changes to the production table because the **Update Prices** task failed.
18. In **SQL Server Data Tools**, double-click the **Update Prices** task and change the **SQLStatement** property to **UPDATE ProductionTable SET Price = 100**. Click **OK**.
19. Run the package and note that all tasks succeed.

20. Stop debugging and close SQL Server Data Tools.
21. In **SQL Server Management Studio**, select the top 1,000 rows from the **dbo.ProductionTable** table, noting that it now contains products with valid prices.
22. Close SQL Server Management Studio.

## Using Checkpoints

Another way you can manage data consistency is to use checkpoints. You can use checkpoints to restart a failed package after the issue that caused it to fail has been resolved. Any tasks that were previously completed successfully are ignored, and the execution resumes at the point in the control flow where the package failed. While checkpoints do not offer the same level of atomic consistency as a transaction, they can provide a useful solution when a control flow includes a long-running or resource-intensive task that you do not wish to repeat unnecessarily—such as downloading a large file from an FTP server.

- Checkpoints allow failed packages to be restarted without repeating previously successful tasks
- Enable checkpoints by setting package properties:
  - CheckpointFileName
  - CheckpointUsage
  - SaveCheckpoints

Checkpoints work by saving information about work in progress to a checkpoint file. When a failed package is restarted, the checkpoint file is used to identify where to resume execution in the control flow. To enable a package to use checkpoints, you must set the following properties of the package:

- **CheckpointFileName.** The full file path where you want to save the checkpoint file.
- **SaveCheckpoints.** A Boolean value used to specify whether the package should save checkpoint information to the checkpoint file.
- **CheckpointUsage.** An enumeration with one of the following values:
  - **Always:** The package will always look for a checkpoint file when starting. If none exists, the package will fail with an error.
  - **Never:** The package will never use a checkpoint file to resume execution and will always begin execution with the first task in the control flow.
  - **IfExists:** If a checkpoint file exists, the package will use it to resume where it failed previously. If no checkpoint file exists, the package will begin execution with the first task in the control flow.

## Demonstration: Using a Checkpoint

In this demonstration, you will see how to use a checkpoint.

### Demonstration Steps

Use a Checkpoint

1. Ensure you have completed the previous demonstrations in this module.
2. Start **SQL Server Management Studio** and connect to the **MIA-SQL** database engine instance using Windows authentication.

3. In Object Explorer, expand **Databases**, expand **DemoDW**, and expand **Tables**.
4. Right-click **dbo.StagingTable** and click **Select Top 1000 Rows** to verify that it contains product data.
5. Using Notepad, open **Products.csv** in the **D:\Demofiles\Mod07** folder. Note that it contains details for three more products. Then close Notepad.
6. Start **SQL Server Data Tools** and open the **Checkpoints.sln** solution in the **D:\Demofiles\Mod07** folder.
7. In Solution Explorer, double-click **Load Data.dtsx**. Note that the control flow consists of a File System task to create a folder, a second File System task to copy the products file to the new folder, and a Data Flow task that loads the data in the products file into the staging table.
8. Click anywhere on the Control Flow surface to select the package, and press F4.
9. In the Properties pane, set the following properties:
  - **CheckpointFileName**: D:\Demofiles\Mod07\Checkpoint.chk
  - **CheckpointUsage**: IfExists
  - **SaveCheckpoints**: True
10. Set the **FailPackageOnFailure** property for all three tasks in the control flow to **True**.
11. On the **Debug** menu, click **Start Debugging** to run the package and note that the **Load to Staging Table** task fails.
12. On the **Debug** menu, click **Stop Debugging**.
13. In the **D:\Demofiles\Mod07** folder, note that a file named **Checkpoint.chk** has been created, and that the File System tasks that succeeded have created a folder named **Data** and copied the **Products.csv** file into it.
14. In **SQL Server Data Tools**, view the **Data Flow** tab for the **Load to Staging Table** task, and double-click the **Derive Columns** transformation. Change the expression for the **NewPrice** column to **100**, and click **OK**.
15. View the **Control Flow** tab, and then on the **Debug** menu, click **Start Debugging**. Note that the **Create Folder** and **Copy File** tasks, which succeeded previously, are not re-executed. Only the **Load to Staging Table** task is executed.
16. Stop debugging, and verify that the **Checkpoint.chk** file has been deleted now that the package has been executed successfully.
17. In SQL Server Management Studio, select the top 1,000 rows from the **dbo.StagingTable**, and note that it now contains data six products.
18. Close SQL Server Management Studio and SQL Server Data Tools.

**Check Your Knowledge**

| Question                                                                                                                                                                                                                                                                                                                                                                                        |                                                                                             |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| <p>You have developed an SSIS package containing two Execute SQL Task tasks. The first task extracts data and the second loads a table. A checkpoint is configured for the package. During execution, the second task fails after loading 50 percent of the table data. What will happen when the package is next executed (assuming the error that caused the failure has been rectified)?</p> |                                                                                             |
| Select the correct answer.                                                                                                                                                                                                                                                                                                                                                                      |                                                                                             |
| <input type="checkbox"/>                                                                                                                                                                                                                                                                                                                                                                        | The package will rerun all tasks from the beginning.                                        |
| <input type="checkbox"/>                                                                                                                                                                                                                                                                                                                                                                        | The package will fail.                                                                      |
| <input type="checkbox"/>                                                                                                                                                                                                                                                                                                                                                                        | The package will rerun the second task, loading the remaining 50 percent of the table data. |
| <input type="checkbox"/>                                                                                                                                                                                                                                                                                                                                                                        | The package will rerun the second task from the beginning.                                  |
| <input type="checkbox"/>                                                                                                                                                                                                                                                                                                                                                                        | The package will rerun the first task but skip the second task to avoid another failure.    |

## Lab B: Using Transactions and Checkpoints

### Scenario

You are concerned that, if the Adventure Works ETL data flow fails, it will leave you with a partially loaded staging database. To avoid this and ensure data integrity, you intend to use transactions and checkpoints.

### Objectives

After completing this lab, you will be able to:

- Use transactions.
- Use checkpoints.

Estimated Time: 30 minutes

Virtual machine: **20767C-MIA-SQL**

User name: **ADVENTUREWORKS\Student**

Password: **Pa55w.rd**

### Exercise 1: Using Transactions

#### Scenario

You have created an SSIS package that uses two data flows to extract, transform, and load Internet sales data. You now want to ensure that package execution always results in a consistent data state, so that if any of the data flows fail, no data is loaded.

The main tasks for this exercise are as follows:

1. Prepare the Lab Environment
2. View the Data in the Database
3. Run a Package to Extract Data
4. Implement a Transaction
5. Observe Transaction Behavior

#### ► Task 1: Prepare the Lab Environment

1. Ensure the **20767C-MIA-DC** and **20767C-MIA-SQL** virtual machines are both running, and then log on to **20767C-MIA-SQL** as **ADVENTUREWORKS\Student** with the password **Pa55w.rd**
2. Run **Setup.cmd** in the **D:\Labfiles\Lab07B\Starter** folder as Administrator.

#### ► Task 2: View the Data in the Database

1. Start **SQL Server Management Studio** and connect to the **MIA-SQL** database engine instance by using Windows authentication.
2. In the **Staging** database, view the contents of the **dbo.Customers** and **dbo.InternetSales** tables to verify that they are both empty.

#### ► Task 3: Run a Package to Extract Data

1. Use **SQL Server Data Tools** to open the **AdventureWorksETL.sln** solution in the **D:\Labfiles\Lab07B\Starter\Ex1** folder.
2. Open the **Extract Internet Sales Data.dtsx** package and examine its control flow.

3. Run the package, noting that the **Extract Customers** task succeeds, but the **Extract Internet Sales** task fails. When execution is complete, stop debugging.
4. In **SQL Server Management Studio**, verify that the **dbo.InternetSales** table is still empty, but the **dbo.Customers** table now contains customer records.
5. In **SQL Server Management Studio**, execute the following Transact-SQL query to reset the staging tables:

```
TRUNCATE TABLE Staging.dbo.Customers;
```

#### ► Task 4: Implement a Transaction

1. Configure the **Extract Customer Sales Data** sequence container in the **Extract Internet Sales Data.dtsx** package so that it requires a transaction.
2. Ensure that the **Extract Customers** and **Extract Internet Sales** tasks both support transactions, and configure them so that, if they fail, their parent also fails.

#### ► Task 5: Observe Transaction Behavior

1. Run the **Extract Internet Sales Data.dtsx** package, noting once again that the **Extract Customers** task succeeds, but the **Extract Internet Sales** task fails. Note also that the **Extract Customer Sales Data** sequence container fails. When execution is complete, stop debugging.
2. In **SQL Server Management Studio**, verify that both the **dbo.InternetSales** and **dbo.Customers** tables are empty.
3. View the data flow for the **Extract Internet Sales** task, and modify the expression in the **Calculate Sales Amount** derived column transformation to resemble that shown below:

```
UnitPrice * OrderQuantity
```

4. Run the **Extract Internet Sales Data.dtsx** package, noting that the **Extract Customers** and **Extract Internet Sales** tasks both succeed. When execution is complete, stop debugging.
5. In **SQL Server Management Studio**, verify that both the **dbo.InternetSales** and **dbo.Customers** tables contain data.
6. Close **SQL Server Data Tools** when you have completed the exercise.

**Results:** After this exercise, you should have a package that uses a transaction to ensure that all Data Flow tasks succeed or fail as an atomic unit of work.

## Exercise 2: Using Checkpoints

### Scenario

You have created an SSIS package that uses two data flows to extract, transform, and load reseller sales data. You now want to ensure that, if any task in the package fails, it can be restarted without re-executing the tasks that had previously succeeded.

The main tasks for this exercise are as follows:

1. View the Data in the Database
2. Run a Package to Extract Data
3. Implement Checkpoints
4. Observe Checkpoint Behavior

#### ► Task 1: View the Data in the Database

1. Use **SQL Server Management Studio** to view the contents of the **dbo.Resellers** and **dbo.ResellerSales** tables in the **Staging** database on the **MIA-SQL** database engine instance.
2. Verify that both of these tables are empty.

#### ► Task 2: Run a Package to Extract Data

1. Open the **AdventureWorksETL.sln** solution in the **D:\Labfiles\Lab07B\Starter\Ex2** folder.
2. Open the **Extract Reseller Data.dtsx** package and examine its control flow.
3. Run the package, noting that the **Extract Resellers** task succeeds, but the **Extract Reseller Sales** task fails. When execution is complete, stop debugging.
4. In **SQL Server Management Studio**, verify that the **dbo.ResellerSales** table is still empty, but the **dbo.Resellers** table now contains reseller records.
5. In **SQL Server Management Studio**, execute the following Transact-SQL query to reset the staging tables:

```
TRUNCATE TABLE Staging.dbo.Resellers;
```

#### ► Task 3: Implement Checkpoints

1. Set the following properties of the **Extract Reseller Data** package:
  - **CheckpointFileName**: D:\ETL\CheckPoint.chk
  - **CheckpointUsage**: IfExists
  - **SaveCheckpoints**: True
2. Configure the properties of the **Extract Resellers** and **Extract Reseller Sales** tasks so that, if they fail, the package also fails.

#### ► Task 4: Observe Checkpoint Behavior

1. View the contents of the **D:\ETL** folder and verify that no file named **CheckPoint.chk** exists.
2. Run the **Extract Reseller Data.dtsx** package, noting once again that the **Extract Resellers** task succeeds, but the **Extract Reseller Sales** task fails. When execution is complete, stop debugging.
3. View the contents of the **D:\ETL** folder and verify that a file named **CheckPoint.chk** has been created.

4. In **SQL Server Management Studio**, verify that the **dbo.ResellerSales** table is still empty, but the **dbo.Resellers** table now contains reseller records.
5. View the data flow for the **Extract Reseller Sales** task, and modify the expression to match the following code sample:

```
UnitPrice * OrderQuantity
```

6. Run the **Extract Reseller Sales Data.dtsx** package, noting the **Extract Resellers** task is not re-executed, and package execution starts with the **Extract Reseller Sales** task, which failed on the last attempt. When execution is complete, stop debugging.
7. In **SQL Server Management Studio**, verify that the **dbo.ResellerSales** table now contains data.
8. Close **SQL Server Data Tools** when you have completed the exercise.

**Results:** After this exercise, you should have a package that uses checkpoints, so that execution can be restarted at the point of failure of the previous execution.